

Dass Endsem Rubric

MCQ + Q6 + Q7 Karthik Q8 + Q12 Rudra Q9 Paarth Q10 Agyeya Q11 George Q13 Vijay Q14 Neel Q15 Agrim Q16 Farzana Q17 Aaditya, Aniket, Divijh Q18 Aman, Santhosh, Agyeya (2nd set)

MCQ + Q6 + Q7

1. c)
2. c)
3. b)
4. b)
5. c)
6. Degree to which elements of a module/class are **functionally related** (partial marks will be given accordingly)
7. Not possible for non-trivial software due to **huge input space and time constraints**. (1 mark) Testing is **selective, not exhaustive**. (1 mark)

Q8 - [Rudra]

1 Mark for Choosing Requirement Coverage 2 Marks for Correct Reasoning

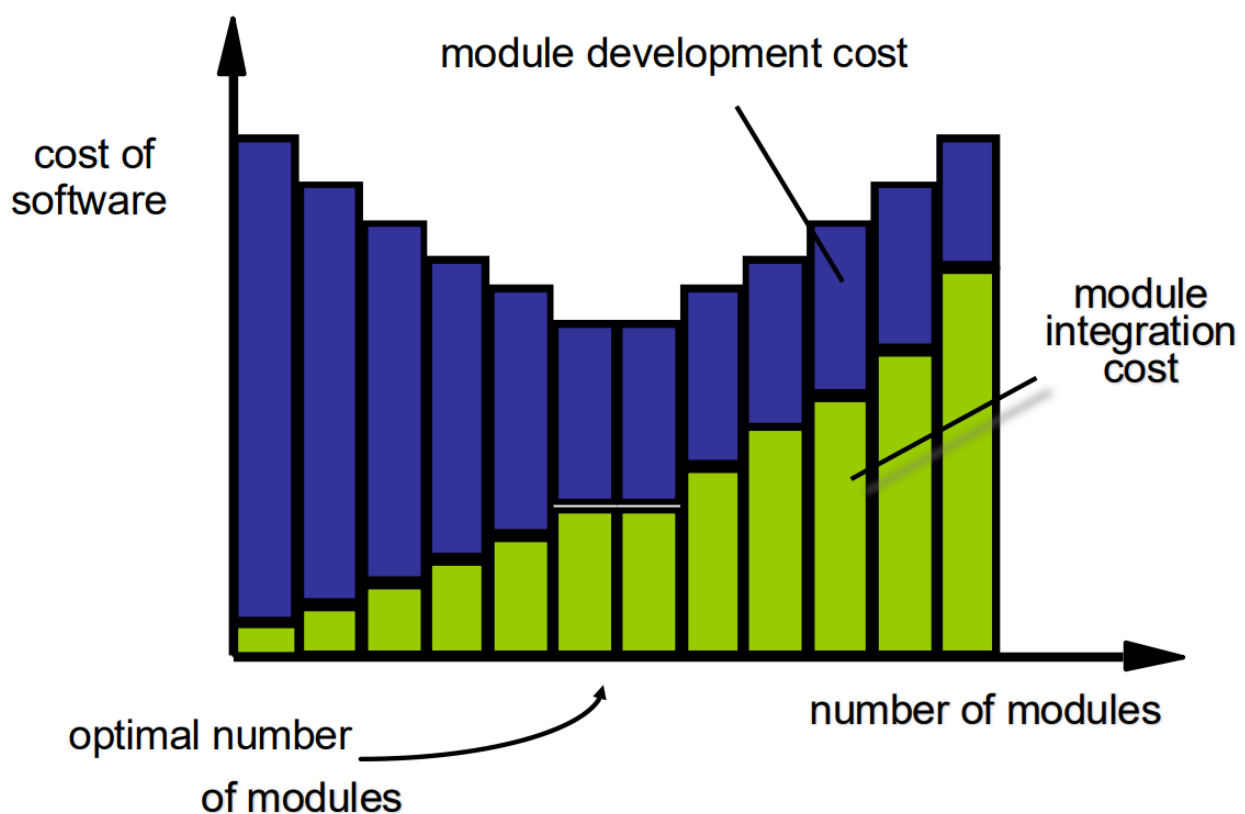
-0.5 Marks if any wrong option is selected

Since no source code is available, it is impossible to use white-box testing methods like statement, function, or path coverage, as these require direct access to the internal code and design. However, because we have documentation explaining the software's functionality, we must use requirements coverage, which is a black-box technique that derives test cases directly from functional descriptions and specifications.

Coverage

- Function coverage: Each function/method executed by at least one test case
- Statement coverage: Each line of code covered by at least one test case (need more test cases than above)
- Path coverage: Every possible path through code covered by at least one test case (need lots of test cases)

Q9 - [Paarth]



1. The Graph (1 Mark)

Score	Criteria
1.0 Mark	Draws a clear U-shaped curve for "Total Software Cost" with both axes correctly labeled (Y-axis: Cost, X-axis: Number of Modules/Modularity).
0.5 Mark	Draws the correct U-shaped curve but misses or incorrectly labels the axes.
0.0 Mark	Missing graph or fundamentally incorrect curve shape (e.g., linear or downward sloping).

2. Detailed Explanation (2 Marks)

A. Individual Module Cost (1 Mark)

- **Full Mark (1.0):** Explains that as the system is broken into more modules, the **internal complexity** of each individual module decreases. Because each piece is smaller and more focused, it becomes significantly easier to design, code, debug, and test. This reduction in "intellectual overhead" for each component directly lowers the individual development cost.
- **Partial Mark (0.5):** Provided in cases where **no explanation** for the decrease is given, reasoning is **wrong**, or the statement is **vague** (e.g., "it is just cheaper to build smaller things").

B. Integration & Interface Cost (1 Mark)

- **Full Mark (1.0):** Explains that as the number of modules increases, the **coordination overhead** grows. This includes the effort required to design and manage complex interfaces (APIs), handle inter-module data communication, and perform extensive integration testing. Complexity shifts from *inside* the modules to the *connections between* them, driving up the cost.
- **Partial Mark (0.5):** Provided in cases where **no explanation** for the increase is given, reasoning is **wrong**, or the statement is **vague** (e.g., "having more parts makes it harder to manage").

Q 10 - [Agyeya]

- **Full Marks (Any 3 mentioned/2 with explanation)**

Key Differences

1.Focus: Sequence diagrams show how objects interact over time. State diagrams show what states an object goes through.

2.Time & Sequence: Sequence diagrams highlight the strict chronological order of messages. State diagrams show the sequence of states, not necessarily time-based interaction.

3.Scope: Sequence diagrams are generally used for specific scenarios or use cases. State diagrams can model the entire lifecycle of a complex object.

4.Elements: Sequence diagrams use lifelines and messages. State diagrams use states, transitions, and events.

Similarities

1. Behavioral Modeling: Both are types of UML diagrams used to model the dynamic aspects of a system.
2. Time-Sensitive: While sequence diagrams are more focused on precise timing, both represent behavior that changes as time progresses.
3. Complementary Purpose: They are often used together to provide a complete picture: the sequence diagram shows how a message causes a transition in a state diagram.

Q11 - [George]

1 Mark: For Bottom Up

Justifications Expected:

discussing the common code and facade class. How common code is critical and has to be tested early. Facade class can be integrated at the last.

module integration helps with verifiability

Sandwich integration is also a valid answer

Function at a time is also a valid answer incase i cut marks please do come to me for papershowing

Have given marks for also saying sometimes like why this one is better than the other one with proper justification

Q12 - [Rudra]

1 Mark For each

(a) Big Bang Integration (b) Top-Down Integration (c) Bottom-Up Integration

Q13 - [Vijay]

Question

A not-very-user-friendly developer is creating software for a library, to help patrons find books. Their user interface design as described below. The text in *italics* is what the program displays to the user. The text in parentheses is our description of how the user interface works.

Step 1 : **Enter the type of search ("title", "subject", "author"):* *_____

(The dashes actually represent a text box into which the user is expected to type in one of the words. If they enter a value other than these three, they get an error message).

Step 2 : **Enter whitespace-separated search terms. Please note that the search is case-sensitive and the relative ordering is used in determining match scores.**_____

(After the user enters the words to search for, the results are displayed. If no matches are found, the

following happens)
 Step 3: (A dialog box pops up): `"Error 203: Bad match. Fix and try again. Press OK to continue"`.
 (The dialog box has only an OK button).
 Step 4: (If they press OK, then the next dialog box pops up.) `"Do you want to do another search?"`):
 There are three buttons on this dialog box: "Yes", "No", "Cancel".
 If the user pushes "Yes", they are taken back to Step 1. If they push "No" or "Cancel", they are taken back to Step 1).

Identify four distinct problems with this user interface design and describe an appropriate solution to each of these problems.

Answer - W10 UI design and usability slide

Step 1 (Text box for exact category):

- Reduce Short-Term Memory (STM) Load: Forcing the user to recall and type exact terms ("title", "subject", "author") instead of recognizing them from a list.
- Rate of User Errors: Using a free-text field for a fixed set of options drastically increases the likelihood of typos and subsequent system errors.

Step 2 (Case-sensitive, relative ordering search):

- Mental Models: Users expect digital searches (like Google or a modern library OPAC) to be forgiving and case-insensitive
- Rate of User Errors: Enforcing strict casing and exact relative ordering makes the system highly unforgiving

Step 3 ("Error 203: Bad match" popup):

- Provide Appropriate Error Handling: The system uses intimidating technical codes ("Error 203") and hostile language ("Bad match") instead of clearly telling the user what went wrong.
- Yield Closure: The interaction ends in a frustrating dead-end without guiding the user toward a successful resolution.

Step 4 ("Yes", "No", "Cancel" all lead to Step 1):

- Support Internal Locus of Control: The user is trapped in an inescapable navigational loop where their choices don't matter
- Consistency: The "No" and "Cancel" buttons do not behave the way these buttons consistently behave in every other software environment.
- Allow Users to Reverse (Undo) Actions: Sending the user back to a blank Step 1 wipes out their previous work, preventing them from simply backing up and editing the typo in their original query.

Rubric Need 4 issues Each issue

- 1.5 mark for each valid issue from above + any valid issues from the slides.
- - 0.5 for not naming the principle violated

- **-0.5** for not mentioning which step causes the violation
-

Q14 - [Neel]

Part (a) (2 points max)

- **2 Points (Full Credit):**
 - **Principle:** Information Expert
 - **Class:** **Order**
 - **Justification:** **Order** holds the necessary data (items, prices, quantities) to calculate the total.
 - *Note: Must include principle, class, and a valid justification.*
 - **1 Point (Partial Credit):** Identifies Information Expert and **Order**, but fails to explain why based on the data.
 - **0 Points (No Credit):** Incorrect principle and class.
-

Part (b) (2 points max)

Award 1 point per correct Principle + Evidence pair, up to a maximum of 2 points.

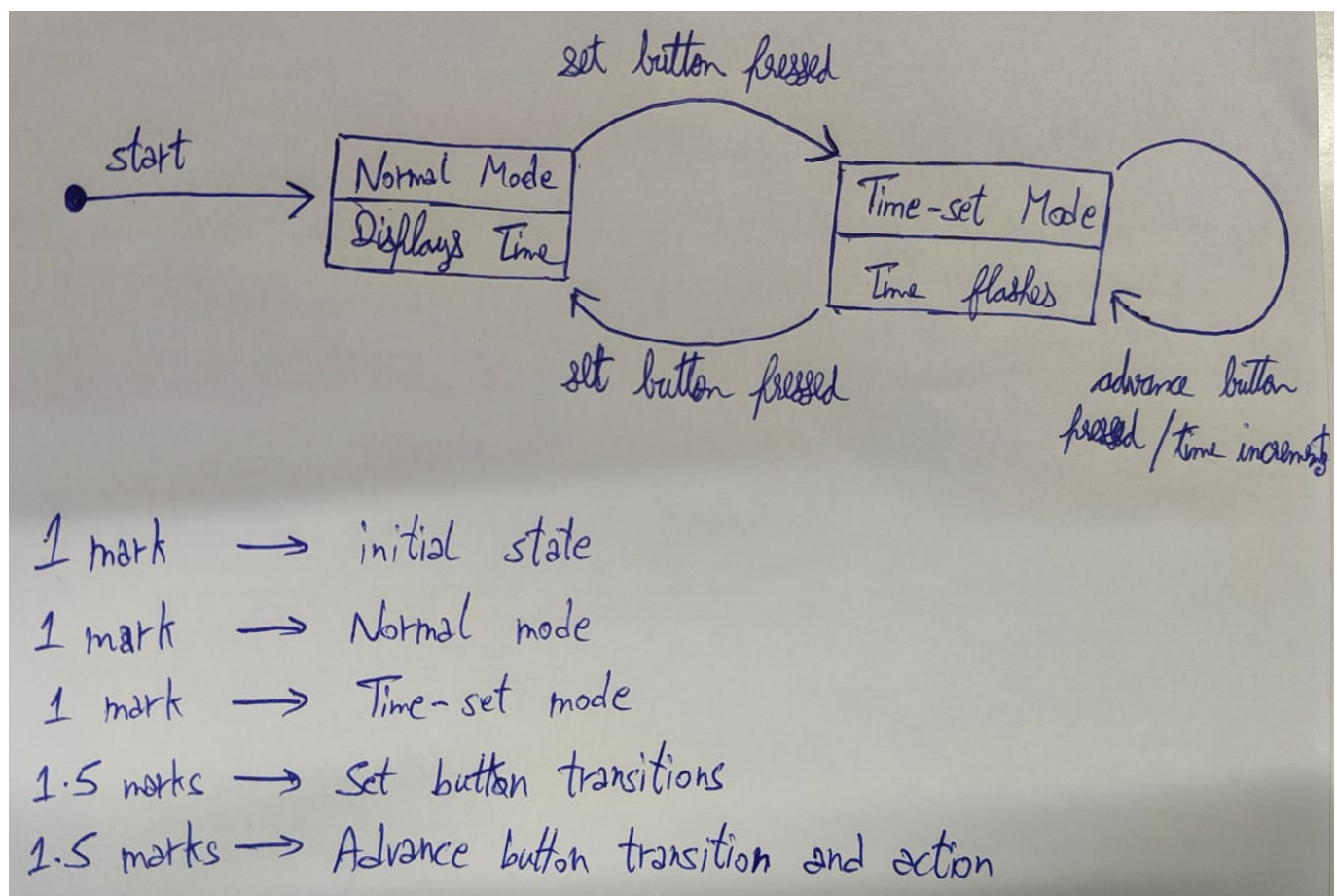
- **1 Point (Option 1):**
 - **Principle:** High Cohesion
 - **Evidence:** The class performs multiple unrelated tasks (saving to DB, sending emails, generating PDFs, math calculations).
 - **1 Point (Option 2):**
 - **Principle:** Low Coupling
 - **Evidence:** Heavily dependent on multiple external objects (**db**, **email_service**, **pdf_gen**) and relies heavily on **Order**'s getter methods.
 - **1 Point (Option 3 - Extra Answer):**
 - **Principle:** Information Expert
 - **Evidence:** Calculates the total by pulling data out of **Order** via getters instead of letting the expert (**Order**) do it.
 - **0 Points (No Credit):** Names a principle but provides no specific evidence from the code snippet.
-

Part (c) (2 points max)

- **2 Points (Full Credit - Standard Expected Answer):**
 - **Primary Principle:** Polymorphism
 - **Target Class:** The **Customer** class (or specific subclasses like **RegularCustomer**, **LoyaltyCustomer**).
 - **Secondary Principle:** Information Expert, Protected Variations, OR Low Coupling.
 - *Note: Must have the correct primary principle, target class, and a valid secondary principle with a brief logical connection.*
- **2 Points (Full Credit - Alternative/Extra Answer):**
 - **Primary Principle:** Pure Fabrication
 - **Target Class:** A newly created, artificial class that does not currently exist in the code snippet.
 - **Secondary Principle:** High Cohesion OR Low Coupling.

- *Note: As long as the student indicates they are creating a dedicated, separate class just to handle the math (which keeps the Customer class highly cohesive), they have successfully identified the correct target class.*
- **1 Point (Partial Credit):**
 - Correct primary principle (Polymorphism) and target, but missing or incorrect secondary connection.
 - *OR (Weak Primary / Strong Logic):* Missed the strict "varies by type" trigger (so they didn't use Polymorphism as primary), but still demonstrated sound architectural logic by using Information Expert or Protected Variations as primary, supported by a valid secondary principle (Polymorphism, Low Coupling, High Cohesion, Protected Variations, Information Expert).
- **0 Points (No Credit):** Incorrect primary principle.

Q15 - Clock State Machine [Agrim]



Q16 - Code Inefficiencies and Improvements [Farzana]

- uses recursion with exponential time complexity [2]
- recalculates the sequence from the start each time [1]
- poor naming of `mystery_function` [1]
- no input validation [1]
- risk of stack overflow / recursion error for large `n` [1]

Q17 — Airline Reservation System (ARS) [Aniket, Divijh]

Total: 12 marks (a) 3 marks · (b) 5 marks · (c) 4 marks

(a) Detailed Textual Use Case (3 marks)

Use Case: Book Reservation

Primary Actor: Passenger

Preconditions:

- Passenger is using ARM (kiosk)
- System is operational
- Flight data already exists in system

Postconditions:

- Reservation created
 - Payment processed
 - Ticket printed
-

Main Success Flow

1. Passenger starts booking on ARM
 2. System displays available routes
 3. Passenger enters source, destination, date
 4. System shows available flights
 5. Passenger selects a flight
 6. System shows seat availability
 7. Passenger selects seat
 8. Passenger selects class (first class / coach)
 9. System displays price
 10. Passenger enters card details
 11. System verifies payment
 12. System creates reservation
 13. System prints ticket
-

Alternate Flows

- **No flights available:** System prompts retry
 - **Seat unavailable:** Passenger selects different seat
 - **Payment failure:** Retry or cancel
-

Marking Scheme

- Structure (actor, pre/post): 1 mark
 - Correct step flow: 1 mark
 - Alternate cases: 1 mark
-

(b) UML Class Diagram (5 marks)

Diagram

```
classDiagram

class Passenger {
    passengerID
    name
    cardInfo
}

class Reservation {
    reservationID
    flightNo
    dateTime
    type
    seatNo
    price
    cancel()
    modify()
}

class Flight {
    flightNo
    origin
    destination
    departureTime
}

class Seat {
    seatNo
    class
    isAvailable
}

class Payment {
    amount
    status
    verify()
}

class Ticket {
    ticketNo
    print()
}
```

```

class ARM {
    machineID
    location
}

class DataClerk {
    clerkID
    enterFlightData()
}

Passenger "1" --> "0..*" Reservation : makes
Reservation "*" --> "1" Flight : for
Flight "1" --> "1..*" Seat : has
Reservation "1" --> "1" Seat : assigns
Reservation "1" --> "1" Payment : uses
Reservation "1" --> "1" Ticket : generates
Passenger --> ARM : uses
DataClerk --> Flight : updates

```

Notes

- Reservation is central class
- DataClerk only interacts with Flight
- Passenger and DataClerk are independent actors

Marking Scheme

- Classes: 2 marks
- Relationships: 1 mark
- Multiplicity: 1 mark
- Attributes/methods: 1 mark

(c) UML Sequence Diagram (4 marks)

Diagram

```

sequenceDiagram

participant Passenger
participant ARM
participant System
participant Flight
participant Reservation
participant Payment
participant Ticket

Passenger->>ARM: startBooking()
ARM->>System: searchFlights()

```

```

System->>Flight: checkAvailability()
Flight-->>System: flightList
System-->>ARM: displayFlights()

Passenger->>ARM: selectFlight()
ARM->>System: createReservation()

System->>Reservation: <>

System-->>ARM: requestPayment()
Passenger->>ARM: enterCard()

ARM->>System: processPayment()
System->>Payment: verify()
Payment-->>System: success

System->>Reservation: confirm()

System->>Ticket: <> + print()
ARM-->>Passenger: ticket

```

Important Notes

- DataClerk is **not part of this sequence**
 - Reservation and Ticket must be **created objects**
 - Payment must occur before confirmation
-

Marking Scheme

- Correct participants: 1 mark
- Correct sequence flow: 1.5 marks
- Message types (call/return): 1 mark
- Object creation: 0.5 mark

Question 18 - Space station [Santhosh]

Sample correct state-diagram

```

stateDiagram-v2
    direction TB

    %% Initial State
    [*] --> Approaching : Vehicle arrives

    %% Entry Branching
    Approaching --> GettingTicket : Driver presses button
    GettingTicket --> WaitingForEntryAirlock : "Ticket dispensed
    [timestamped]"

```

```

    Approaching --> WaitingForEntryAirlock : Driver swipes permit

    %% Entry Airlock Phase
    WaitingForEntryAirlock --> InEntryAirlock/WaitingForGarage : "[airlock_capacity >= vehicle_size] / enter_airlock()"

    InEntryAirlock/WaitingForGarage --> ParkedInGarage : "[garage_count < 100] / enter_garage()"

    %% Exit Branching & Payment Phase
    ParkedInGarage --> PayingAttendant : Ticket holder wants to leave
    PayingAttendant --> WaitingForExitAirlock : "Payment processed / record_payment()"

    ParkedInGarage --> WaitingForExitAirlock : "Permit holder swipes / charge_account()"

    %% Exit Airlock Phase
    WaitingForExitAirlock --> InExitAirlock : "[airlock_capacity >= vehicle_size] / exit_garage(), enter_airlock()"

    %% Final State
    InExitAirlock --> [*] : "Exit to space / exit_airlock()"

```

Rubric

- 1 mark - UML syntax
- 1 mark - Start and End states
 - 0.5 for start(Solid Dot), 0.5 for exit(Bulls eye)
- 2 marks - Core states (Arrival,Airlock,Garage,Exit)
- 2.5 mark - Flow correctness
- 1.5 marks - Payment flow (Ticket->Payment, Permit)
- 2 marks - Constraints (garage_count < 100, airlock_capacity >= vehicle_size)